Tutorial/Laboratory 03

Creating Dynamic, Interactive Websites with JavaScript

1. LEARNING OUTCOMES

Upon completion of these laboratory exercises, you should be able to:

- Implement collapsible, mobile-friendly navigation menus with Bootstrap JS.
- Utilize client-side scripting with JavaScript to build dynamic, interactive websites.
- Debug JavaScript applications using the Chrome or Firefox debuggers.

2. REQUIRED SOFTWARE

- Visual Studio Code (https://code.visualstudio.com/)
- Supported web browser (Chrome or FireFox recommended).
- Bootstrap JS (https://getbootstrap.com/docs/5.3/getting-started/javascript/)

3. GETTING STARTED

3.1 In this practical assignment, we'll continue with the *World of Pets* website that we built in previous labs. Therefore, you should first make a copy of the most recent project and rename it to Lab03.

3.2 In the following exercises, we will improve our website by making it more dynamic and interactive. Namely:

1. Improve the menu by implementing Bootstrap's JavaScript and the collapsible mobile menu.
2. Add event handlers so that when the user clicks on the pet images, we'll display the larger image in a popup window, rather than opening the image in the browser.

4. EXERCISE 1: INSTALLING BOOTSTRAP JAVASCRIPT

4.1 In the previous Lab, we only installed Bootstrap CSS. But Bootstrap also provides JavaScript for enabling some features, such as the collapsible mobile menu. We'll use the CDN method again to load Bootstrap JS. **Note**: be sure to use the same version of Bootstrap JS that you used for Bootstrap CSS (e.g., 5.3.1).

4.2 Copy the following code into the <head> element in index.html, just below the existing <link> element:

```
<!--Bootstrap JS-->
<script defer
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.1/dist/js/bootstrap.bundle.min.js"
    integrity="sha384-HwwvtgBNo3bZJJLYd8oVXjrBZt8cqVSpeBNS5n7C8IVInixGAoxmnlMuBnhbgrkm"
    crossorigin="anonymous">
</script>
```

Note the use of the **defer** attribute in our <script> element. This tells the browser to wait until the complete web page has been loaded and parsed before loading the JavaScript. Before this attribute was added in HTML5, developers typically placed the <script> element at the end of the document, just before the closing </body> tag.

Discussion Questions:

a. Does it matter when JavaScript files are loaded?

b. What are some of the ramifications of loading JavaScript before the HTML document is fully parsed and displayed?

c. What are the advantages of putting all <script> elements in the <head>?

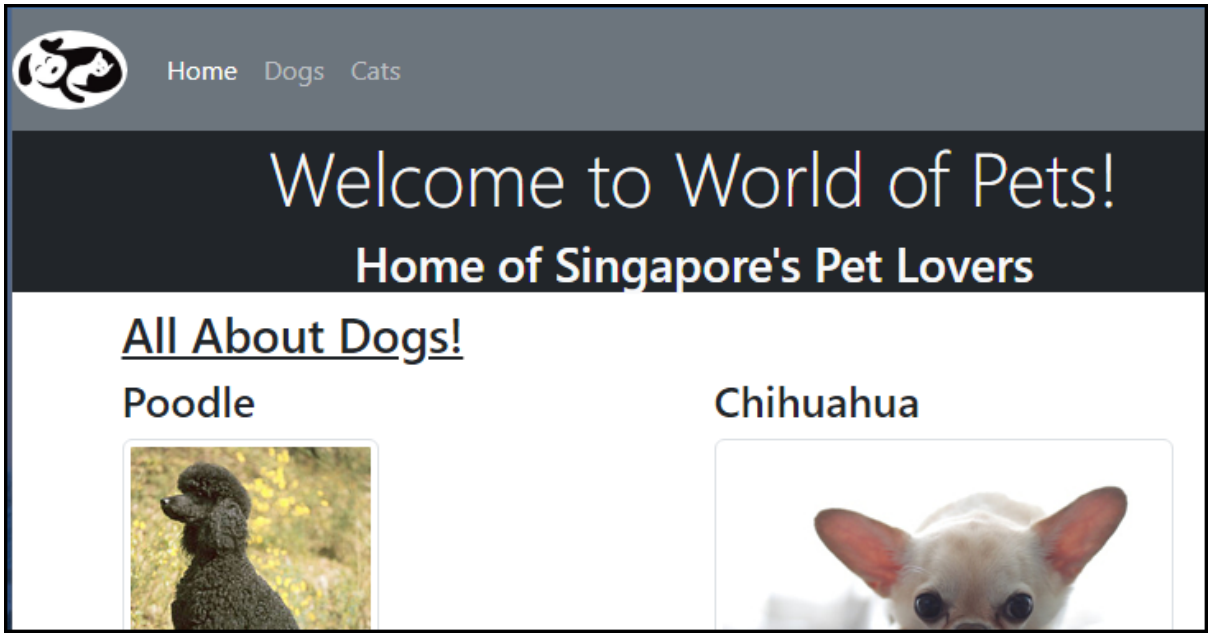
d. What does the **async** attribute do and how is it different from **defer**?

5. <u>EXERCISE 2: IMPLEMENTING THE COLLAPSIBLE MOBILE MENU</u>

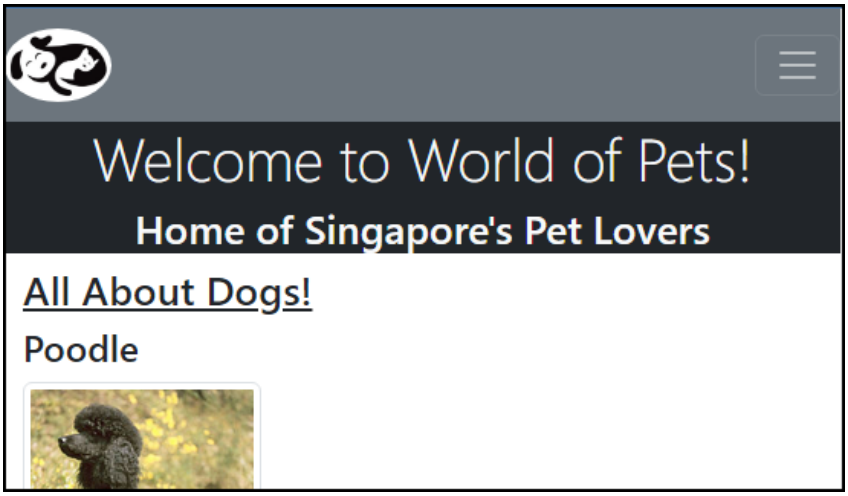
5.1 To make our navigation menu more responsive and usable on mobile devices, we'll take advantage of Bootstrap's collapsible menu feature.

Refer to the examples given at: https://getbootstrap.com/docs/5.3/components/navbar/ then modify your existing <nav> element so that when the browser is resized past the small break point, the menu collapses to the mobile version. Example:

Normal menu:



Mobile menu:



6. EXERCISE 3: JAVASCRIPT EVENT LISTENERS

6.1 One of the primary uses of client-side scripting is to respond dynamically to events that occur while the user is interacting with the web page. In this exercise, we'll demonstrate this by adding JavaScript *event listeners* that respond to the user clicking on the pet article images. Instead of opening the larger image in the browser, we'll display it in a popup window. Then when the user clicks the image again, the popup will close.

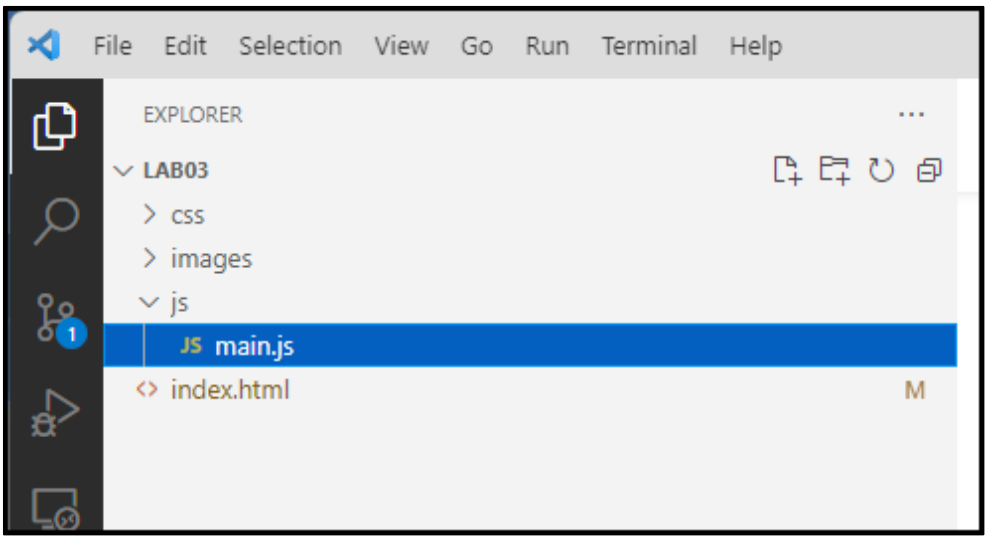
It is up to you how you accomplish this, but here are some suggested steps you may want to follow:

a. Remove the existing <a> elements that wrap each <img> element.

b. Add a class attribute (e.g. class="img-thumbnail" or similar) to each <img> tag so that your JavaScript function can select only the images that should have a popup when clicked. You could have other images in the page for which you do *not* desire the popup effect, so the class attribute is a handy way to designate the ones that should.

c. Edit your external CSS file (main.css) and add a ruleset for styling the popup images. You can do this by adding a new class, say ".img-popup", which will be dynamically applied to the popup image in your JavaScript code. Add property-value pairs for styling items such as the image size, border, background, padding, position, etc. Refer to https://developer.mozilla.org/en-US/docs/Web/CSS for tips on CSS styling.

d. Add a custom external JavaScript file (e.g. “main.js”) to your project. You should put this in a sub-folder called **js** that’s just below the project root folder, similar to how you added your custom CSS file previously:



Don’t forget to include your JavaScript file in index.html by adding another <script> element below the existing ones:

```
<!-- Custom JS -->
<script defer src="js/main.js"></script>
```

e. Edit main.js and add a “click” event listener to all elements of the img-thumbnail class.
Tips:

- You’ll want the code that registers your event listeners to be executed automatically when the page first loads. To do this, you can add an event listener to the document object that listens for the “DOMContentLoaded” event. Then when that event fires, you can add event listeners for any other elements on the page. Example:

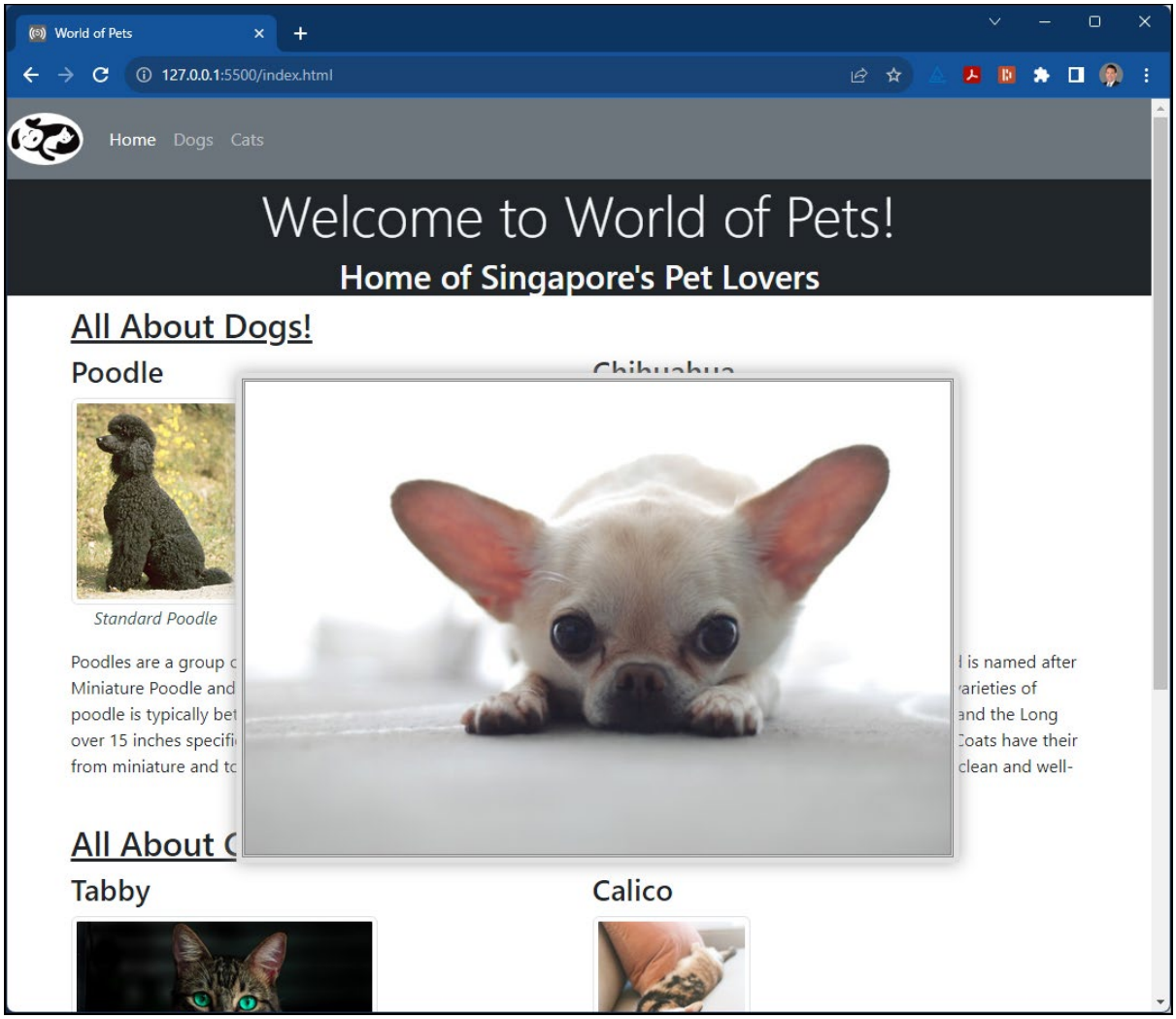
```
document.addEventListener("DOMContentLoaded", function ()
{
    // Code to be executed when the DOM is ready (i.e. the document is
    // fully loaded):
    registerEventListners(); // You need to write this function…
});
```

- You can make use of the standard getElementsByClassName() JavaScript function to get an array of all <img> elements in that class, then loop through the array to add the event listener to each one.

- To dynamically create the popup image, you can use the createElement() JavaScript function (a **span** element should work fine). Then use setAttribute() to set the "img-popup" class to the new element, and the innerHTML property to add the new <img> element within the .
- Lastly, to get the newly created element to show up in the web page, you must call a function to add the element to the DOM. There are a few different functions in JavaScript you can use, for example, insertAdjacentElement(), which inserts the new element next to a sibling element (so that they share the same parent).
- You'll also need some logic to check for and remove any existing popup image.

You may refer to https://www.w3schools.com/js/js_htmldom_eventlistener.asp for more details on event listeners, and to the sample code provided for you on xSiTe.

Example showing popup after clicking on the Chihuahua image:



There are many ways to achieve this and you are welcome to experiment and come up with your own solutions. The main point is to learn how to set event listeners and make your page more interactive with JavaScript.

Discussion Questions:

a. Why do we prefer to use the event listener method as opposed to the older inline JavaScript event handlers (e.g. “onclick=” attribute in the HTML element)?

b. What are some of the advantages of putting your JavaScript code in an external file?

c. What happens if the user has disabled JavaScript or it’s not supported by the device/browser? As a web developer, how should you prepare for that?

7. EXERCISE 4: DEBUGGING JAVASCRIPT

7.1 For anyone wishing to make their career in software development, debugging skills are invaluable. JavaScript applications are especially prone to errors and learning how to properly debug them will save you countless hours and headaches.

7.2 For this exercise, go to https://developer.chrome.com/docs/devtools/javascript/ and run though the tutorial on debugging JavaScript in Chrome. Although this is specific to Chrome, the debugger in FireFox and other browsers work very similarly, and the concepts are the same.

Note: there is no submission required for the debugging exercise, however you are responsible for knowing the concepts, including:

- Setting breakpoints
- Stepping through JavaScript code
- Inspecting variable values
- Using the console

8. SUBMISSION OF LAB ASSIGNMENT

8.1 In order to receive credit for this Lab assignment, you must submit your completed work to xSiTe LMS before the end of the Lab session. To submit your work:

a. Save all files and close Visual Studio.

b. In File Explorer, navigate to the location where you saved your project and right-click on the folder name, then select 'Send to -> Compressed (zipped) folder' and ZIP up your entire project. **Note: only .zip format is acceptable, do not use .rar, .7z, or any other format.**

c. In the INF1005 module on xSiTe, go to **Assessments->DropBox** and locate the Dropbox folder corresponding to this Lab. Click the link to open the Dropbox then hit the **Add a File** button to submit your .zip file. You may also add comments if desired. Be sure to hit **Submit** to complete your submission.

d. Remember to save a copy of your work as we will be building upon this website in subsequent Lab assignments.

9. ADDITIONAL PRACTICE

9.1 Once you've completed this Lab assignment, you are encouraged to check out the following online references:

a. JavaScript: https://www.w3schools.com/js/default.asp